**Laborator 4**

ORM - Code First

Pentru interacțiunea cu baza de date utilizând conceptul Code First, este nevoie de configurarea mai multor lucruri.

În primul rând se creează fișierul ***appsettings.json,*** unde configurăm connection string-ul care se folosește pentru conectarea la baza de date.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=DocumentStorageDB_CodeFirst;User Id=sa;Password=reallyStrongPwd123;TrustServerCertificate=True;"
  }
}
```

**Figura 1 -** Configurarea connection string-ului

După ce am configurat connection string-ul, este nevoie să înregistrăm contextul bazei de date în fișierul ***Program.cs***, cum vedem în figura 2.

```
builder.Services.AddDbContext<DocumentStorageDbContextCodeFirst>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString( name: "DefaultConnection")
    ));
```

**Figura 2 -** Înregistrarea contextului

Apoi, după ce am configurat conectiunea la baza de date, putem efectua prima migrare de creare a bazei de date. În scopul evitării conflictelor cu baza existentă, am creat o a doua bază de date pentru efectuarea acestui laborator. În figura 3, vedem comenzile rulate pentru crearea și aplicarea unei migrări asupra bazei de date.

Atașat voi pune fișierul migrărilor atât pentru crearea inițială, cât și pentru o modificare ulterioară a unei coloane.

```
(base) bogdanboicu@Mac DocumentStorage.Data % dotnet ef migrations add InitialCreate --context DocumentStorageDbContextCodeFirst -s ../DocumentStorage.Server
Build started...
Build succeeded.
Done. To undo this action, use 'ef migrations remove'
(base) bogdanboicu@Mac DocumentStorage.Data % dotnet ef database update --context DocumentStorageDbContextCodeFirst -s ../DocumentStorage.Server
Build started...
Build succeeded.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (219ms) [Parameters=[], CommandType='Text', CommandTimeout='60']
      CREATE DATABASE [DocumentStorageDB_CodeFirst];
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (162ms) [Parameters=[], CommandType='Text', CommandTimeout='60']
      IF SERVERPROPERTY('EngineEdition') <> 5
      BEGIN
```

**Figura 3 - Comenzile de crearea și aplicare a unei migrări**

Pentru a practica conceptul Code First, propun să modificăm clasa AccountUser, și să adăugăm o coloană nouă, *EmailAddress*. În figura 4, vedem structura actuală pe care o avem în cod, și în baza de date a tabelului dat.

```
using ...

namespace DocumentStorage.Data.Models;

[PrimaryKey( propertyName: "AccountId",  params additionalPropertyNames: "UserId")]
public partial class AccountUser
{
    [Key]
    public Guid AccountId { get; set; }

    [Key]
    public Guid UserId { get; set; }

    [StringLength(50)]
    public string Role { get; set; } = null!;

    [ForeignKey( name: "AccountId")]
    [InverseProperty("AccountUsers")]
    public virtual Account Account { get; set; } = null!;

    [ForeignKey( name: "UserId")]
    [InverseProperty("AccountUsers")]
    public virtual User User { get; set; } = null!;
}
```

**Figura 4 - Structura clasei AccountUser**

```
[StringLength(50)]
public string EmailAddress { get; set; } = null!;
```

**Figura 5 - Proprietatea adăugată**

Prin intermediul migrării, adăugăm schimbarea în baza de date.

```
> C# 20260112125803_AddedEmailAddressToAccountUser.cs
```

S-a creat migrarea, și s-a aplicat în baza de date. În figura 6 putem vedea că în tabelul AccountUser s-a adăugat proprietatea EmailAddress.

| AccountId | UserId | Role | EmailAddress |
|---|---|---|---|

**Figura 6 -** Coloanele tabelului AccountUser.

În urma aplicării migrării vedem că schimbările din cod al entității, s-au reflectat și în tabelul din baza de date.